CS 4475 Fall 2026
Shirley Moore, Instructor
Md Jahangir Alam, TA

PA2: Priority-based Scheduler for xv6
80 points

For this assignment, you will implement a priority-based scheduler for xv6. xv6 currently uses a round-robin scheduler. You should keep the round-robin scheduler and add your priority-based scheduler so that one or the other may be used by setting the appropriate values in param.h to select the scheduler at compile-time. For process effective priorities, use a range of integers from 0 to 99, with processes allowed to have priorities 0 to 49 and effective priorities 0 to 99. The init process should start running with priority 0. A child process should inherit its parent's priority. A process should be able to query its priority by using the getpriority() system call and set its priority using the setpriority() system call. Your priority scheduler should always select a process with the highest priority as the next process to run. Your priority scheduler should increase the effective priority of a process by using aging. The "age" of a process will be determined by how long it has been waiting in the ready queue. Rather than maintaining an explicit age field in struct proc, it will be more efficient to store the time at which the process became ready to run, say in a field called readytime, and calculate the effective priority from the priority and readytime fields. You may decide on your own aging policy. A simple example could be the following:

```
#define MAXEFFPRIORITY 99
effective_priority = min(MAXEFFPRIORITY, priority + (currtime - readytime))
```

Important note: For this homework and the last one, we want to use just one CPU so that all processes are scheduled on the same CPU. To make this happen, in the Makefile, change CPUS := 3 to CPUS := 1.

Create a pa2 branch in your xv6 repository with

```
git checkout -b pa2
```

You can also merge in your pa1 branch so that you have the time command as well.

Task1. (15 pts) You are given the code for a ps command. Add the following struct to kernel/pstat.h:

```
struct pstat {
  int pid;     // Process ID
   enum procstate state;  // Process state
   uint64 size;     // Size of process memory (bytes)
   int ppid;       // Parent process ID
   char name[16];   // Parent command name
};
```

Add the following prototype under systems calls in user/user.h:

```
int getprocs(struct pstat*);
```

Then implement the getprocs() system call that returns the current number of processes and fills in the array of struct pstat that is passed into the callas an argument. Test your getprocs() system call by running the ps command.

Task 2. (15 pts) Add a priority field to struct proc and implement the getpriority() and setpriority() system calls. Don't forget to initialize the priority field in allocproc(). Modify the getprocs() system call and ps program to also retrieve and print the priority field. Write a test program and use your modified ps command to check that your new system calls work and show the results.

Task 3. (15 pts) Add a readytime field to struct proc and initialize it to the current time whenever the process's state gets changed from another state to RUNNABLE. Add the readytime field to struct pstat in kernel/pstat.h and modify the ps command to print the process's age if the process state is RUNNABLE, where the age is the current time minus readytime.

Task 4. (15 pts) Add constants to param.h to choose between scheduling policies at compile time. Implement a priority-based scheduler that selects the highest priority process to run next. If there is a tie, select any highest priority process. Write some test programs to verify that your priority scheduler works as expected and show the results.

Task 5. (15 pts) Decide on an aging policy and add aging to your priority scheduler. Write some test programs to verify that your aging policy works as expected and show the results. Evaluate your results in terms of average turnaround time and response time.

Sample output

pexec is a user program that takes a priority, a command, and the command's arguments as inputs and runs the command at that priority.
Your output need not look exactly like this.

```
$ pexec 5 matmul 5 &; matmul 10 &
$ pexec 10 ps
pid  state      size     age   priority  cputime  ppid  name
1    sleeping   12288          0         0        0     init
2    sleeping   16384          0         0        1     sh
29   sleeping   12288          10        0        2     pexec
28   runnable   12288   1      0         8        1     matmul
26   sleeping   12288          5         0        1     pexec
27   runnable   12288   0      5         55       26    matmul
30   running    12288          10        0        29    ps
$ pexec 10 ps
```

```
pid  state      size     age   priority  cputime  ppid  name
1    sleeping   12288          0         0        0     init
2    sleeping   16384          0         0        1     sh
31   sleeping   12288          10        0        2     pexec
28   runnable   12288  2       0         16       1     matmul
26   sleeping   12288          5         0        1     pexec
27   runnable   12288  0       5         104      26    matmul
32   running    12288          10        0        31    ps
$ pexec 10 ps
Time: 161 ticks
pid  state      size     age   priority  cputime  ppid  name
1    sleeping   12288          0         0        0     init
2    sleeping   16384          0         0        1     sh
33   sleeping   12288          10        0        2     pexec
28   runnable   12288  0       0         59       1     matmul
34   running    12288          10        0        33    ps
$ Time: 404 ticks
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you encountered and if/how you overcame them.

Extra Credit Task. (10 pts) Implement getpriority and setpriority commands that get and set the priority of the current sh process, respectively. Since a child process inherits its parent's priority, these commands will also affect the priority of any command executed by sh. (**Hint**: Remember what was discussed in class about the cd and exit commands).

Sample output:

```
$ getpriority
0
$ setpriority 10
$ getpriority
10
$ ps
pid  state      size     age   priority  cputime  ppid  name
1    sleeping   12288          0         0        0     init
2    sleeping   16384          10        0        1     sh
3    running    12288          10        0        2     ps
```

Turn-in procedure and Grading

Please use the accompanying template for your report. Convert your report to PDF format and push your pa2 branch with your code and report to your xv6 GitHub repository by the due date**.** Also turn in the assignment on Teams with the URL of your github repo by the due date. Give access to your repo to the instructor and TA.

This assignment is worth 80 points. The breakdown of points is given in the task descriptions above. The points for each task will be evaluated based on correctness of your code, proper coding style and adequate comments, and the section of your report for that task. Grading will be done during a required demo.

You may discuss the assignment with other students, but do not share your code. Your code and lab report must be your own original work. Any resources you use should be credited in your report. If we suspect that code has been copied from an online website or github repo, from a book, or from another student, or generated using AI without giving appropriate credit and describing in detail how AI was used, we will turn the matter over to the Office of Student Conduct for adjudication.